Matthew Zeng
mz22535
April 30, 2026

COE 311K Final Project

Part 1 - Numerical Solution of Second-Order ODEs

Section 1: Introduction & Model System

Real-World System: This project will analyze a spring-mass-damper (SMD) system, such as those found in car suspensions. SMD systems have a mass attached to a linear coil spring and a damper, which result in dampened oscillations when they experience disturbances.

Governing Second-Order ODE: The motion equation for SMD systems is

$$m\ddot{y} + c\dot{y} + ky = 0$$

where *m* is the mass (kg), *c* is the damping coefficient (N·s/m), and *k* is the spring constant (N/m). *y* is the displacement from equilibrium (m). Solving for the acceleration $\ddot{y}$, we obtain

$$\ddot{y} = -\frac{c}{m}\dot{y} - \frac{k}{m}y$$

Conversion to First-Order ODE System: Let

$$y_1 = y$$

$$y_2 = \dot{y}.$$

The system then becomes

$$\dot{y}_1 = y_2$$

$$\dot{y}_2 = -\frac{c}{m}y_2 - \frac{k}{m}y_1$$

Section 2: Parameter Research & Justification

Real-World Scenario: We will analyze the suspension of 2023 Toyota Camry LE (without AWD) driving over a neighborhood speed bump at low speeds. The curb weight is around 3310 lb (1501 kg), the speed bump height is around 3 in (0.0762 m), and the driving speed is around 10 mph (16.09 kph).

Parameter Values:

Parameter	Symbol	Value	Units	Source
Mass per spring	m	375.35	kg	Camry curb weight divided by 4
Spring constant	k	30,000	N/m	Standard MacPherson strut from Camry datasheet
Damping ratio	ζ	0.32	—	Standard MacPherson strut from Camry datasheet

Matthew Zeng
mz22535
April 30, 2026

Initial displacement	y_0	0.0762	m	Typical neighborhood speed bump height
Vehicle velocity	v_{car}	16.09	kph	Determined by arbitrary scenario
Tire radius	r	0.334	m	Tire diameter converted to radius in meters
Simulation time	t	3.0	s	Enough time to return to equilibrium

Derived Parameters:

Parameter	Symbol	Formula	Value	Units
Natural frequency	ω_n	$\sqrt{\frac{k}{m}}$	8.94	rad/s
Damped frequency	ω_d	$\omega_n\sqrt{1-\zeta^2}$	8.47	rad/s
Damping coefficient	c	$2\zeta\sqrt{mk}$	2147.63	N·s/m
Initial vertical velocity	$\dot{y}_0$	$v_{car} \cdot sin(arctan(\frac{y_0}{r}))$	3.58	kph

Sources:

1. https://www.toyota.com/content/dam/toyota/brochures/pdf/2023/camry_ebrochure.pdf
2. https://highways.dot.gov/safety/speed-management/traffic-calming-eprimer/module-3-part-2#3.10
3. https://www.bfgoodrichtires.com/auto/tires/bfgoodrich-advantage-control?maker=Toyota&model=Camry&segment=LE+AWD

Section 3: Numerical Implementation

Implementation Approach: Both methods use the same function *f* to maintain consistent comparison between the models. The damping coefficient *c* is calculated from the damping ratio (obtained from sources), and the initial vertical velocity of 3.58 kph uses an accurate but semi-approximate formula using the car's velocity, speed bump height, and tire radius for slow speeds and standard bump heights.

Matthew Zeng
mz22535
April 30, 2026

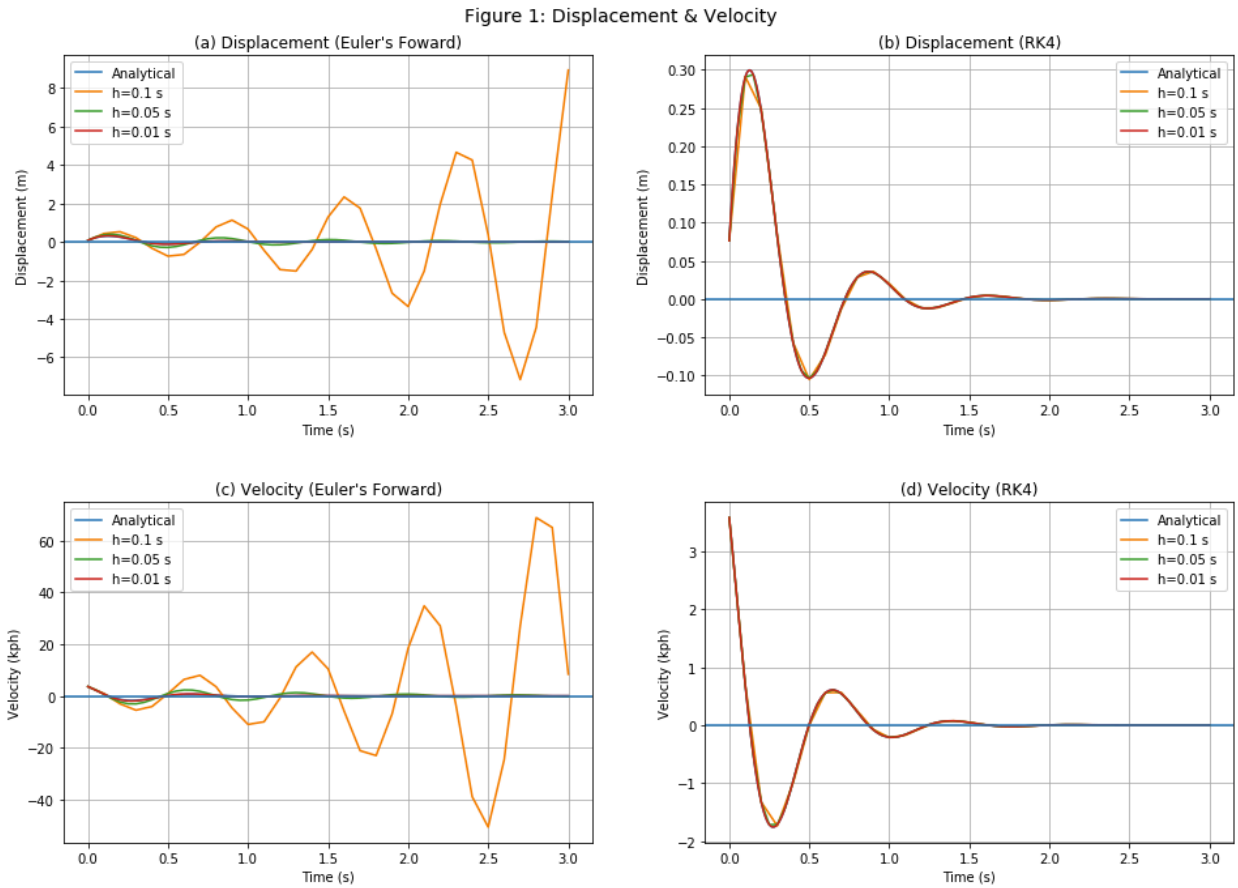

Figure 1. Displacement and velocity graphs for Euler's Forward and RK4 methods using different step sizes

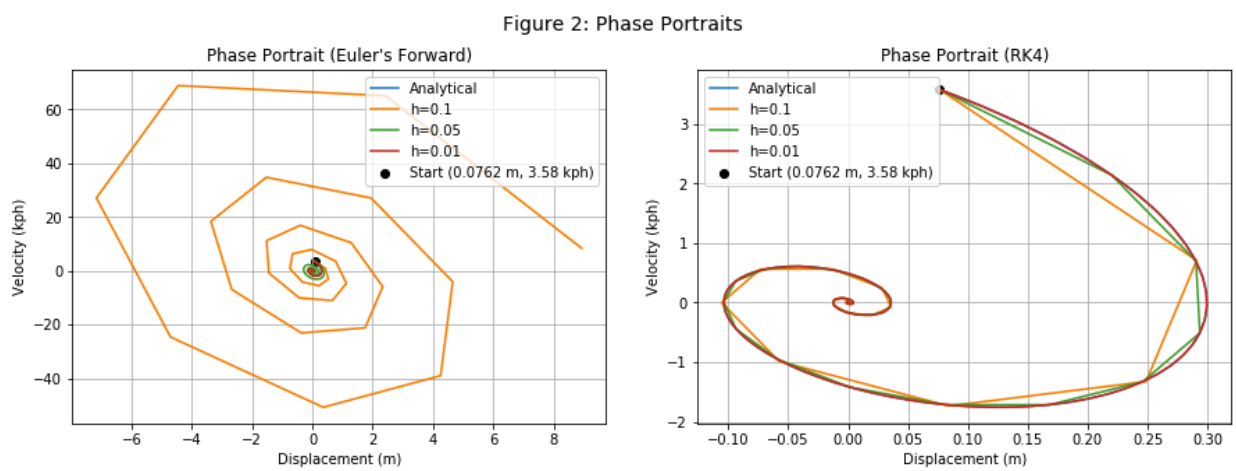

Figure 2. Phase portraits for Euler's forward and RK4 methods with initial displacement at 0.0762 m and initial velocity 3.58 kph

Matthew Zeng
mz22535
April 30, 2026

Section 4: Solutions & Comparisons

Discussion: Figure 1 models the suspension the moment the car hits the speed bump at 16.09 kph. The displacement rises slightly over the initial displacement, as the car's initial momentum carries it a little higher past the actual height of the bump. Since the system is underdamped with the damping ratio being 0.32, there are a few bounces before the car returns to equilibrium. The RK4 method follows the analytical solution more than Euler's forward at all step sizes. At h = 0.1, Euler's method is unstable and blows up, while it improves as h decreases to 0.01. Figure 2 shows inward spiraling from the initial state toward the origin. RK4 follows the spiral very accurately while Euler's drifts slightly and spirals outward at h = 0.1 due to energy input from errors.

Section 5: Stability Analysis

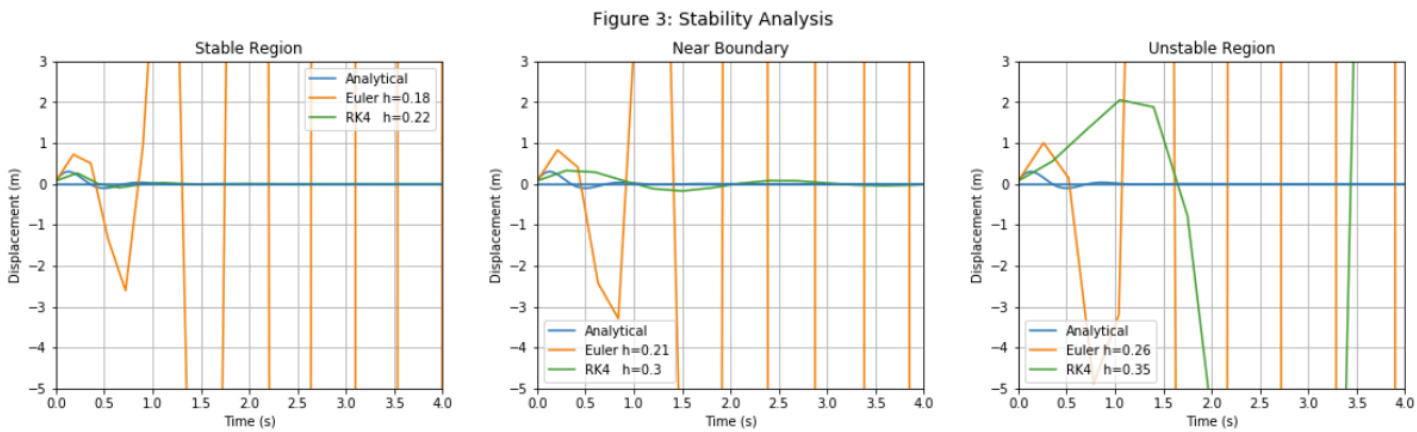

Figure 3. Stability analysis of Euler's method and RK4 for step sizes around their respective stability limits (Euler: 0.224; RK4: 0.32)

Step Size Investigation: The Euler stability limit 0.224 and the RK4 stability limit is 0.32. For both solutions, the model stays stable up until their respective limits and begins blowing up past their limits. However, RK4 simply tolerates larger step sizes before becoming unstable, as seen in Figure 3. These instabilities indicate that the car continues to oscillate increasingly, which is impossible.

Matthew Zeng
mz22535
April 30, 2026

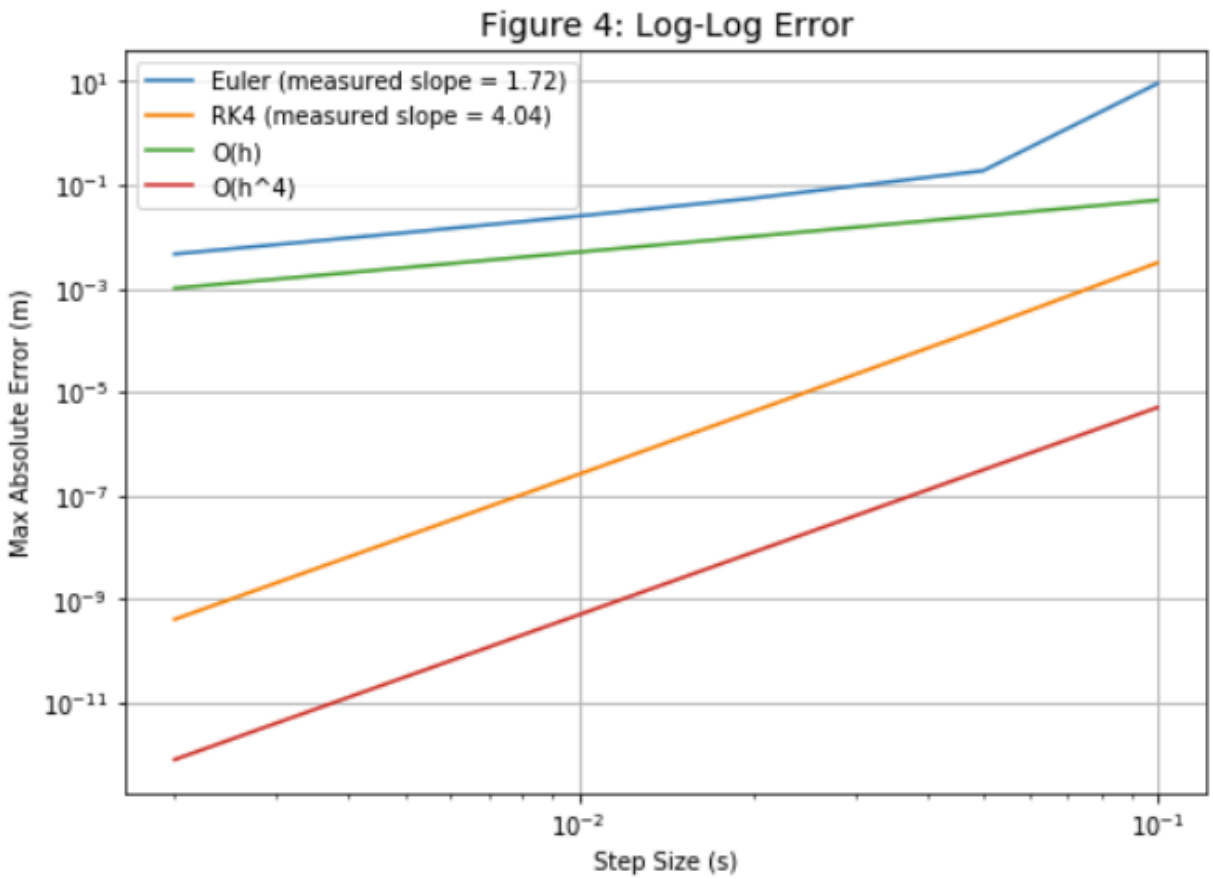

Figure 4. Log-log convergence plot of error vs step size, showing convergence of rates of both methods compared to O(h) and O(h^4) references

Error Quantification: Euler's O(h) convergence causes each halving of h to reduce the error by 2 times, while RK4's O(h^4) convergence makes each halving of h reduce the error by $2^4 = 16$ times. This is seen in Figure 5's measured slopes, which are close to their expected values.

Matthew Zeng
mz22535
April 30, 2026

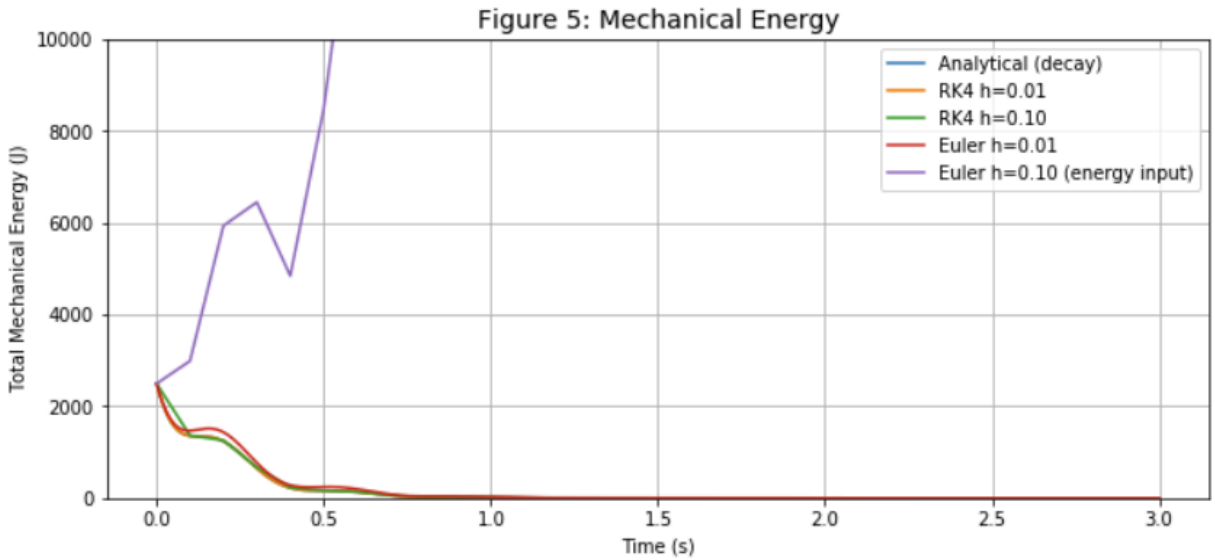

Figure 5. Total mechanical energy of the system over time (must decay due to dampening)

Physical Interpretation: The total mechanical energy is expected to decrease due to damping, as the shock absorber in the suspension cannot introduce more energy to the system. At h = 0.1, Euler's method seems to add energy at each step, which is due to small errors that eventually add up and inject energy into the system. RK4 stays very close to the analytical decay of energy along each step.

Section 6: Conclusion

Method Comparison: RK4 performed much better than Euler's Forward method in essentially all step sizes. Looking at Figure 2, the phase portraits of RK4 all converge toward the origin, while at h = 0.1, Euler's method blows up and has visible drift at the other step sizes.

Computation Cost vs Accuracy: RK4 requires 4 function evaluations per step while Euler's only uses 1. However, this means that RK4 can achieve the same accuracy that Euler's method does at 10x the step size, which actually makes it computationally cheaper for some nominal accuracy goal. For our 3-second model, RK4 at h = 0.01 (1,200 computations) outperforms Euler's which requires h = 0.001 (3000 computations) for the same accuracy.

Practical Recommendations: It would be optimal to use RK4 at h = 0.01 for standard suspension systems (like the one analyzed). Try to avoid using Euler's method for oscillatory systems, as the energy additions due to error misleads physical interpretations along with its more restrictive stability limit compared to RK4. For small step sizes, Euler's method is viable, but RK4 would still be more optimal in all cases if computationally permitted.

Matthew Zeng
mz22535
April 30, 2026

Lessons Learned: Be as exact with parameters as possible and understand the physical scenario before inputting parameters. Energy analysis provides physical context to the errors of Euler's method, which helps with physical interpretation compared to just plots alone. Stability does not necessarily guarantee accuracy, as with Euler's method, even being well below the stability limit can still lead to artificial energy injection. Stability is important but not enough for physical accuracy.

Matthew Zeng
mz22535
April 30, 2026

Part 2 - Stiff ODEs and Implicit Methods

Section 1: Introduction & System Selection

Real-World System: This project will analyze a first-order chemical reaction regarding a concentration that rapidly decays toward a slowly varying equilibrium. We will solve the stiff ordinary differential equation (ODE) for this model, which commonly appears in chemistry patterns.

Governing ODE: The ODE for the chemical reaction is

$$\frac{dC}{dt} =- k(C - C_{eq})$$

with variables and parameters:

Variable	Value	Units	Meaning
k	10000	s^{-1}	Reaction rate constant
C(0)	0	mol/L	Initial concentration
C_{eq}	$1 + 0.1sin(0.1t)$	mol/L	Slowly varying equilibrium concentration
t	60	s	Simulation time

Physical Interpretation: The chemicals C react and approach the equilibrium C_{eq} at rate k. The very fast reaction changes the concentration toward C_{eq} while C_{eq} slowly oscillates. Some real-world applications include fast enzyme kinetics and combustions.

Analytical Solution: Since the ODE is linear, an exact solution exists because of our parameters. The particular solution $y_p(t)$ accounts for the slow but forced oscillation, and the homogeneous solution e^{-kt} models the initial fast decay from the initial condition with

$$y(t) = [y(0) - y_p(0)] \cdot e^{-kt} + y_p(t), \text{ where}$$

$$y_p(t) = \frac{1+0.1[ksin(\omega t)-\omega cos(\omega t)]}{k^2+\omega^2}$$

Because k is much greater than ω, the $y_p(t)$ approximates to C_{eq}, meaning the solution is very close to equilibrium after the initial quick reaction.

Matthew Zeng
mz22535
April 30, 2026

Section 2: Demonstrating Stiffness

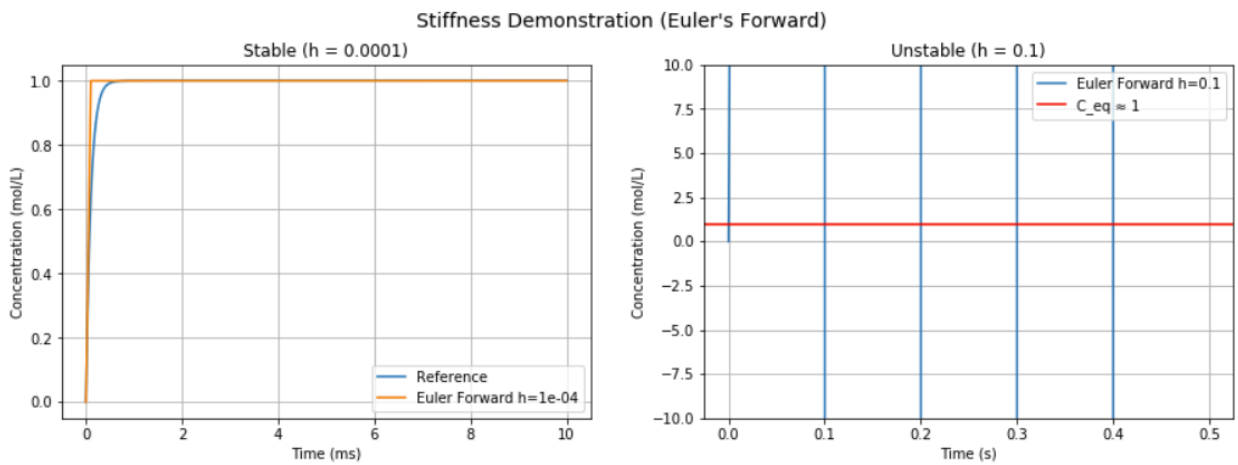

Figure 1. Euler's Forward with h = 0.0001 (left) remains barely remains stable and Euler's Forward with h = 0.1 (right) oscillates violently

Stiffness Ratio: For our analyze ODE, the only eigenvalue is $\lambda = -k = -10,000$. The stiffness ratio is calculated as the ratio between the magnitudes of the eigenvalue and a reasonable time step. Because we have only one eigenvalue and our time step is h = 0.1, we obtain a stiffness ratio of 10,000/0.1 = 100,000.

Impracticality: For Euler's Forward method, the theoretical stability limit is h = 2/k = 0.0002. To simulate a full 60 seconds would require 60/0.0002 = 300,000 steps and computations in total, which is extremely computationally burdensome. If we want to use a more reasonable step size of h = 0.1, Euler's Forward method blows up instantaneously, proving that the system is stiff and not suitable for our problem.

Section 3: Mathematical Setup

Residual Function: For our ODE, the implicit Euler's Backward is

$$C_{n+1} = C_n + h \cdot f(t_{n+1}, C_{n+1}).$$

Because C_{n+1} is part of the function f, there is no explicit solution, forcing us to use the residual function G(C_{n+1}) = 0 at each step. For our case, the residual function is

$$G(C) = C - C_n - h \cdot f(t_{n+1}, C).$$

Substituting $f = -k(C - C_{eq})$, we obtain

$$G(C) = C - C_n - h(-k \cdot \ ]C - C_{eq}(t_{n+1})]), \text{ or}$$

$$G(C) = C \cdot (1 + hk) - C_n - hk \cdot C_{eq}(t_{n+1})$$

Derivation of Residual: The derivative of our function with respect to C is given by

Matthew Zeng
mz22535
April 30, 2026

$$\frac{dG}{dC} = 1 + hk.$$

We see that this derivative is constant since the ODE is linear with respect to C. Since we know that the function is linear, Newton Raphson (NR) can solve the equation with one iteration, as approximating a linear function with a linear model is exact. If the derivative depended on C, it would take multiple iterations since the function would be curved, and linear models would not be exact.

Newton-Raphson Solver: Solving for C_{i+1}, we obtain

$$C_{i+1} = C_i - \frac{G(C_n)}{\frac{dG}{dC}}, \text{ or}$$

$$C_{i+1} = C_i - [C_i(1 + hk) - C_n - hk \cdot C_{eq}] / (1 + hk)$$

with a tolerance of $|C_{i+1} - C_i| < 10^{-10}$ and initial guess of

$$C_0 = C_n + h \cdot f(t_n, C_n).$$

Matthew Zeng
mz22535
April 30, 2026

Section 4: Implementation

```python
# Parameters
k   = 1e4
C0  = 0.0
T   = 60.0

# C_eq
def C_eq(t):

    return 1.0 + 0.1 * np.sin(0.1 * t)

# dCeq_dt
def dCeq_dt(t):

    return 0.01 * np.cos(0.1 * t)

# f
def f(t, C):

    return -k * (C - C_eq(t))
```

```python
def newton_raphson(t_next, C_n, h, tol=1e-10, max_iter=20):

    # G(C) = C - C_n - h*(-k*(C-C_eq(t_{n+1}))
    #      = C(1 + hk) - C_n - hk*C_eq(t_{n+1}) = 0

    # dg/dC = 1 + hk (constant)

    Ceq_next = C_eq(t_next)
    dGdC = 1.0 + h * k

    # initial guess
    C = C_n + h * f(t_next - h, C_n)

    iterations = 0
    for i in range(max_iter):
        G = C - C_n - h * f(t_next, C)
        C_new = C - G / dGdC
        iterations += 1
        if abs(C_new - C) < tol:
            return C_new, iterations
        C = C_new

    # max iterations reached
    return C, iterations
```

```python
def euler_backward(h, T=T, C0=C0):
    t = np.arange(0, T + h, h)
    C = np.zeros(len(t))
    C[0] = C0
    iters = []  # Newton-Raphson iterations per step

    for n in range(len(t) - 1):
        C[n+1], it = newton_raphson(t[n+1], C[n], h)
        iters.append(it)

    return t, C, np.array(iters)
```

Figure 2. Code for parameters and ODE (top left), Newton Raphson solver (top right), and Euler's Backward method (bottom)

Matthew Zeng
mz22535
April 30, 2026

Section 5: Results & Analysis

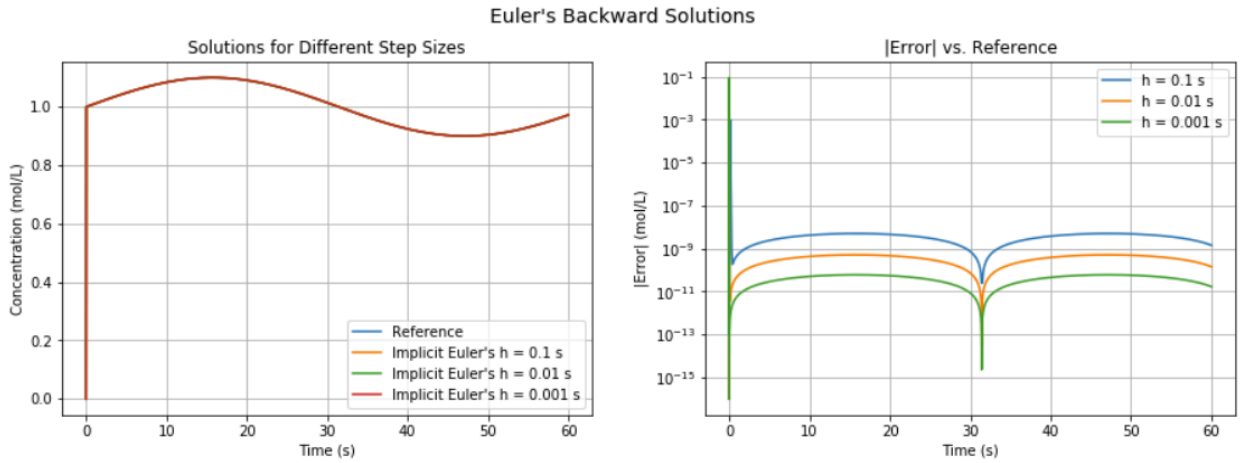

Figure 3. For different steps, Euler's Backward compared to analytical reference(left) and absolute error of reference plotted against time (right)

Solutions for Multiple Step Sizes: Euler's Backward method models the analytical solution very accurately regardless of step size even though the largest step (h = 0.1) is 10^3 times larger than the stability limit for Euler's Forward. However, as h decreases, the errors decrease proportionally, even though they're essentially unnoticeable.

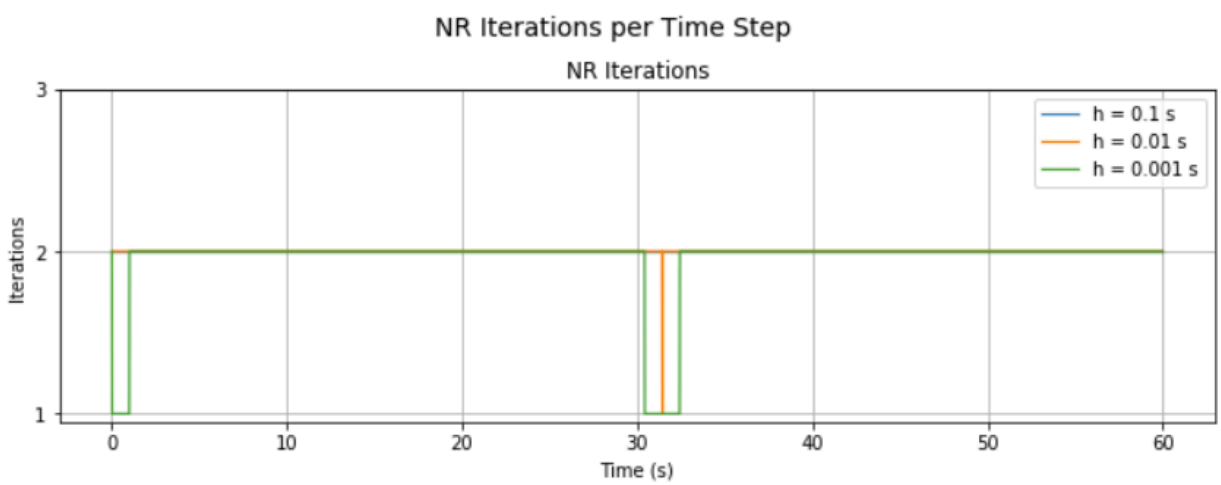

Figure 4. Newton Raphson iterations for each time step (either 1 or 2), showing that the method converges almost immediately for the system

Newton Raphson Convergence: NR should theoretically converge in 1 time step due to the linear nature of the ODE. However, looking at Figure 4, we see 2 iterations because of the portion of our code that checks for convergence, which essentially adds another iteration.

Matthew Zeng
mz22535
April 30, 2026

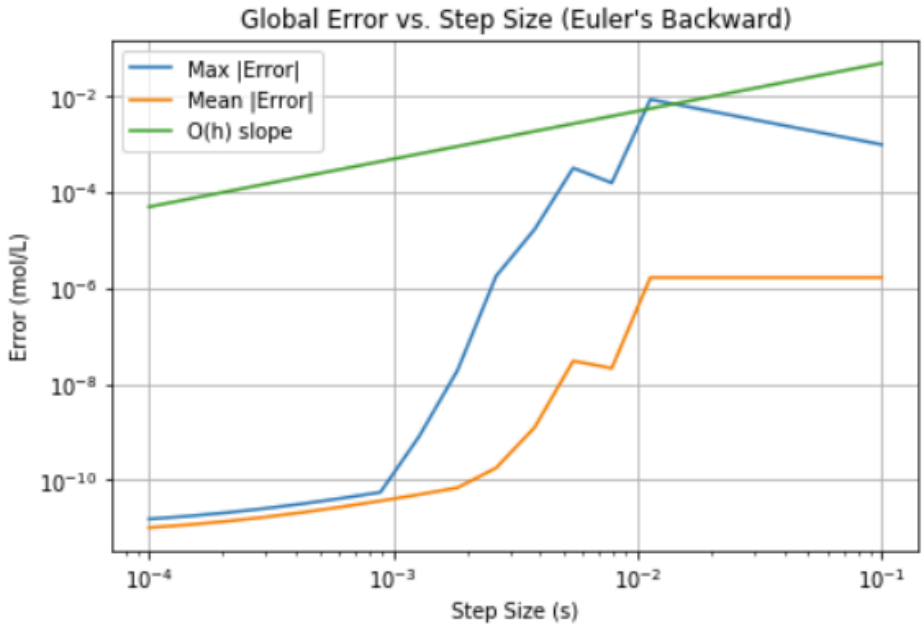

Figure 5. Global mean and max errors compared to O(h) slope for Euler's Backward method on log-log plot

Error vs Step Size: The error plot largely follows the slope of the O(h) slope, confirming that Euler's Backward is a first-order ODE. Although there are a few minor bumps, spikes, or oscillations, the overall slope is roughly the same, meaning that the error scales by the same amount the step sizes change.

Section 6: Performance Analysis

Method	Step Size	Steps	NR Iterations	Total Evaluations	Stable?
Euler's Forward	0.0001	600,000	1	600,000	Yes
Euler's Forward	0.1	600	1	600	No
Euler's Backward	0.1	600	2	2,400	Yes
Euler's Backward	0.01	6,000	2	24,000	Yes
Euler's Backward	0.001	60,000	2	234,000	Yes

Accuracy-Efficiency Tradeoff: For Euler's Backward, a step size h = 0.1 generates a maximum error of around 0.001 mol/L with only 600 steps. Further lowering h to 0.001 reduces the error 100x while also requiring 100x more steps, the overarching tradeoff that must be considered.

Matthew Zeng
mz22535
April 30, 2026

For most applications such as our fast chemical reaction, an error of 0.001 mol/L is fitting, and we can optimize computational burdens.

Practical Recommendation: For our system, a good choice would be to use Euler's Backward method at a step size *h* within the range of 0.01-0.1 (exact choice determined by desired accuracy). This method can provide 100-1,000x less computational burden than Euler's Forward method, which forced us to use a step size of h = 0.0001 for stability.

Section 7: Conclusions

Key Findings:

- Stiffness is less of an accuracy problem and more of a stability problem. The stiffness ratio of our system is 10^5, which is way too high and requires 600,000 evaluations over the course of 60 seconds.
- Euler's Backward method solves our stiffness issue. By calculating a recursive equation at each step, the model remains stable, allowing it to use step sizes 10^3 larger than the stability limit of Euler's Forward while still generating accurate solutions.
- Newton-Raphson converges in 1 iteration for our linear ODE since the derivative is constant. This implies that each time step takes one solve while a nonlinear ODE would take multiple solves.

Implicit vs Explicit: Use implicit solutions when the stiffness ratio is much larger than 1. If the magnitude of the fastest eigenvalue is many orders of magnitude higher than the desired step size, use implicit methods to avoid instability. If the system is not stiff, explicit methods are generally adequate and faster for each step.

Challenges and Lessons: For demonstrating stiffness, choosing a step size that was not too unstable required a little experimentation. The simplified analytical solution was also needed as a reference since solving the integral for many time steps is unrealistic. Errors also scale with O(h), so error scales the same as step sizes, confirming first-order convergence.

Matthew Zeng
mz22535
April 30, 2026

Part 3 - Damping and Adaptive Step Sizes

Section 1: Review of Part 2

In part 2, we modeled a first-order ODE for a very quick chemical reaction with a rate constant of k = 10,000 s^{-1} that decays toward a slowly varying equilibrium $C_{eq}(t) = 1 + 0.1sin(0.1t)$. The governing ODE is given by

$$\frac{dC}{dt} = -10^4 C + 10^4(1 + 0.1sin(0.1t)).$$

We found the stiffness ratio to be $|\lambda|/\omega = 10^4/0.1 = 10^5$, which forces Euler's Forward to use a step size of $h = 2 \cdot 10^{-4}$ purely for stability, resulting in 600,000 evaluations for 60 seconds of simulation. To combat the stability problem, we implemented Euler's Backward with Newton-Raphson at a more reasonable step size of $h = 0.1$.

Section 2: Newton-Raphson Damping Implementation

Newton-Raphson Damping: An ordinary Newton-Raphson (NR) can overshoot the initial guess of the root when solving $C^{i+1} = C^i - \frac{G(C^i)}{G'(C^i)}$, which causes the residual to increase and eventually diverge.

Line Search: We can modify the equation to obtain

$$C^{i+1} = C^i + \alpha\frac{G(C^i)}{G'(C^i)}$$

where the damping factor $\alpha \in (0, 1]$ is chosen by a line search algorithm. This algorithm: **1.)** computes the Newton factor **2.)** Sets $\alpha = 1$ to attempt a full step **3.)** Test to determine whether residual decreases **4.)** If it decreases, we accept the step. If not, we reduce the step until it decreases.

Matthew Zeng
mz22535
April 30, 2026

```python
def damped_newton_raphson(C_n, t_next, h, tol=1e-10, max_iter=20, alpha_min=1e-4):
    # computer newton factor
    # start with alpha = 1
    # halve alpha until residual decreases
    # accept updated step

    C = C_n + h * f(t_next - h, C_n)

    alpha_history    = []
    total_backtracks = 0

    for i in range(max_iter):
        g = G(C, C_n, t_next, h)

        if abs(g) < tol:
            return C, i + 1, True, alpha_history, total_backtracks

        # newton factor
        delta = -g / dGdC(h)

        # line search
        alpha = 1.0
        g_abs = abs(g)
        backtracks = 0

        for _ in range(20):
            C_trial = C + alpha * delta
            g_trial = abs(G(C_trial, C_n, t_next, h))

            if g_trial < g_abs:
                break

            alpha /= 2.0
            backtracks += 1

            if alpha < alpha_min:
                alpha_history.append(alpha)
                total_backtracks += backtracks
                return C, i + 1, False, alpha_history, total_backtracks

        alpha_history.append(alpha)
        total_backtracks += backtracks
        C = C + alpha * delta

    return C, max_iter, False, alpha_history, total_backtracks
```

Figure 1. Code for Newton-Raphson with damping and line search

Matthew Zeng
mz22535
April 30, 2026

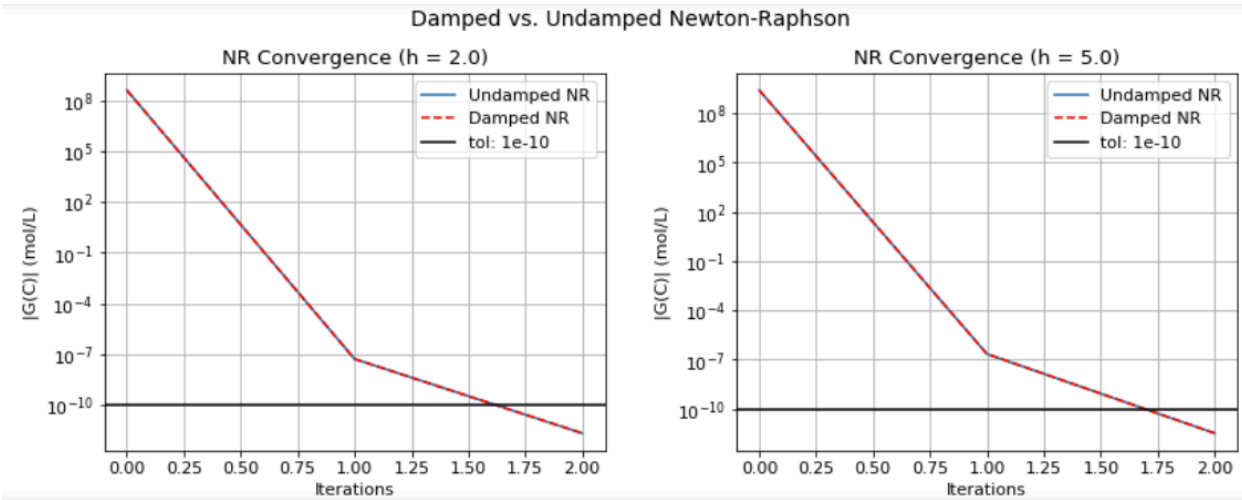

Figure 2. Newton-Raphson residual convergence for damped and undamped systems at h = 2.0 (left) and h = 5.0 (right)

Damped vs Undamped: There is no difference between damped and undamped NR convergence regardless of step size. Both models converge toward our chosen tolerance. This is due to the fact that the linear system will always converge in 1 iteration, regardless of damping or step size.

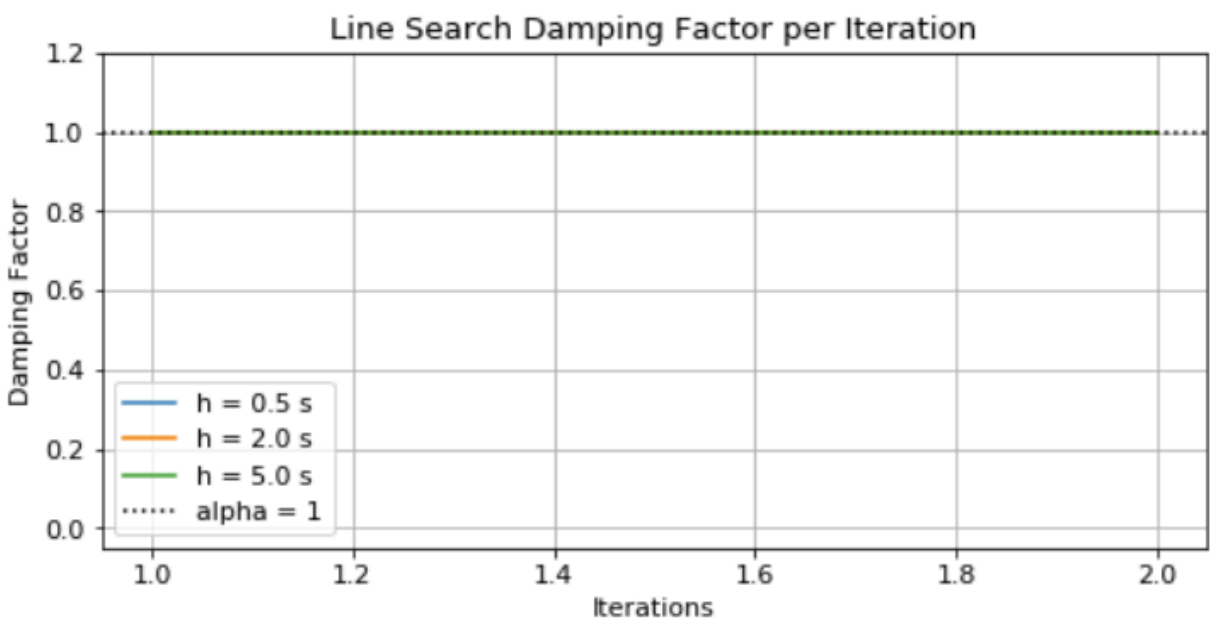

Figure 3. Damping factor used at each iteration for 3 different step sizes.

Necessity of Damping: Damping becomes necessary if the ODE is nonlinear. Because we are using a linear ODE, the Newton factor is exact each time. For all step sizes of a linear ODE, damping is largely unnecessary because of the 1-iteration convergence.

Matthew Zeng
mz22535
April 30, 2026

Section 3: Adaptive Step Size Implementation

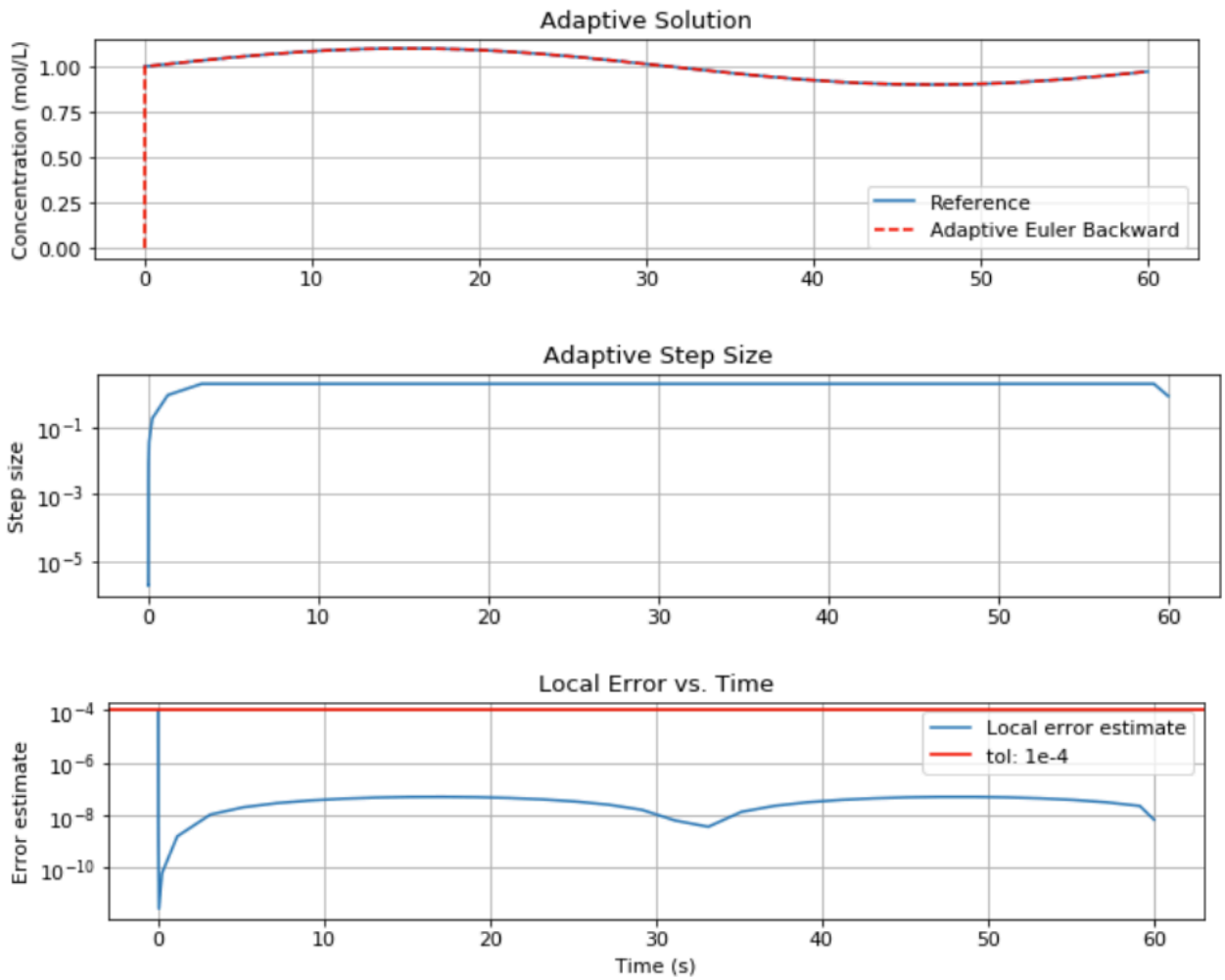

Figure 4.. Adaptive Euler's Backward compared to approximated reference (top). Plot of adaptive step size on a log scale which quickly expands to h_{max} (middle). Local error estimate of solution stays below tolerance (bottom)

Step Doubling Error Estimation: We can estimate the error due to truncation by comparing two different ways to get from t_n to $t_n + h$: **1.)** Single step of size h **2.)** Single step of size h/2. Since Euler's Backward has O(h) local error, a single step of size h has error C(h) while a step of size h/2 has error C(h/2). The error of each step is given by

$$err = |y_{n+1}^{h/2} - y_{n+1}^{h}|.$$

When we use half steps, each new step size is determined by

$$h_{new} = safety \cdot h\sqrt{\frac{tol}{err}}$$

Matthew Zeng
mz22535
April 30, 2026

with safety = 0.9 to avoid any overly aggressive changes while the change rate is bounded between 0.2h and 5h to prevent small oscillations or large jumps. We also enforce a minimum bound $h_{min} = 10^{-6}$ and a maximum bound $h_{max} = 2$.

Step Size Adaptation: The initial concentration C(0) = 0 is far from the equilibrium concentration $C_{eq}(0) = 1$ at the beginning. The ODE forces a very quick rise in concentration in around 10^{-4} seconds. However, since we start at a step size h = 0.5, the first step skips over the transient reaction and ends up near the analytical solution. Since this error estimate is large, we reject the step and reduce it until our error is under tolerance. After around 10 steps, our solver accepts the entirety of the solution of a slowly oscillating C_{eq} that increases h to $h_{max} = 2$.

Section 4: Performance Analysis

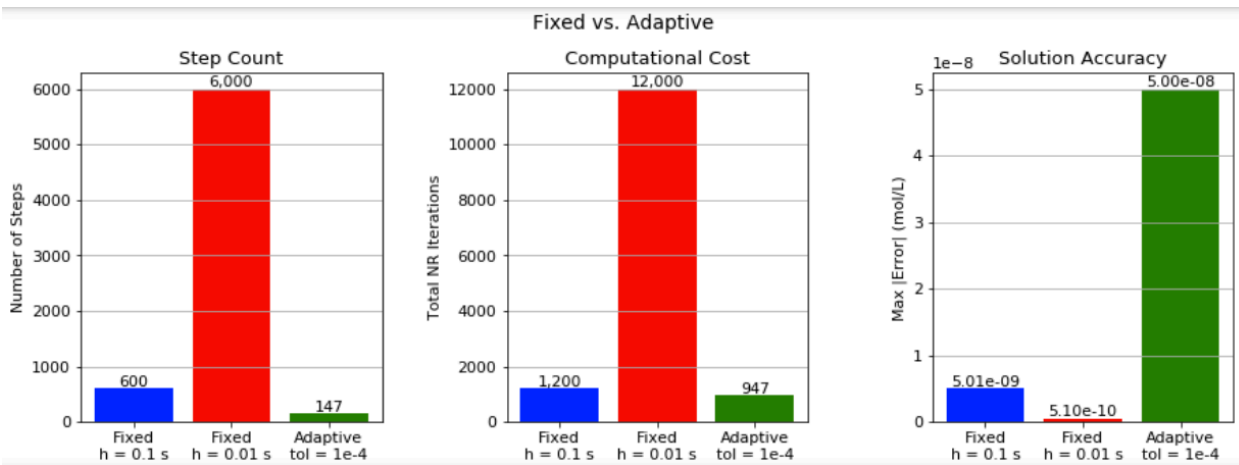

Figure 5. Bar charts comparing total accepted steps (left), total NR iterations (middle), and maximum absolute error (right) between two fixed solutions at different step sizes and an adaptive solution

Fixed vs Adaptive: We compare 3 strategies over a simulation time of 60 seconds, which are all also compared against the analytical reference solution after the quick transient reaction.

Method	h(s)	Accepted Steps	Total NR Iterations	Max Error
Fixed	0.1	600	1,200	5.0e-09
Fixed	0.01	6,000	12,000	5.1e-10
Adaptive (tol=1e-4)	0.03-2.0	147	947	5.0e-08

Accuracy-Efficiency Tradeoff: The fixed solution reaches maximum error in 1,200 iterations. The adaptive solution reaches maximum error in 947 iterations. For the same number of iterations, the adaptive solution could have either a tighter tolerance or reach a higher accuracy than the

Matthew Zeng
mz22535
April 30, 2026

fixed. Although the fixed model at h = 0.01 reaches a smaller error, it takes over 12x the computational cost than the adaptive.

Cost Analysis: Each adaptive step requires 3 evaluations (1 full step and 2 half steps) compared to the 1 fixed step. Despite more computations, the adaptive solver uses a total of 147 steps instead of 600, so the total solves are actually 147 x 3 = 441. The adaptive solution is also more optimal due to its ability to safely use a step size of h = 2 while a fixed solution must use a step size h < 0.1. Although the step-doubling increases the steps by 3x, the total number of steps more than offsets those losses.

Section 5: Robustness Testing

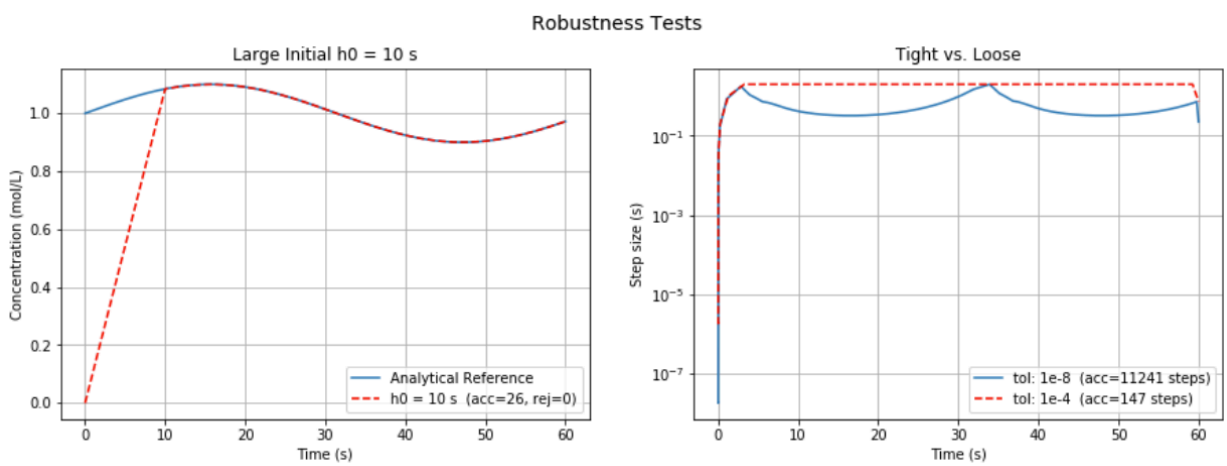

Figure 6. Testing the solver with a very large initial step size (left) that converges to the analytical reference and a tight tolerance (right) that converges to the same step size as a looser tolerance but starts slowly oscillating

Test 1 (Large Initial h_0 = 10): The adaptive solution is initialized with h_0 = 10, a step size 50x the size of the equilibrium oscillation constant. The step-doubling error estimate notices the large initial step size and rejects new h values until step sizes are small enough. The solver then adjusts upward and calculates the correct solution in 26 accepted steps.

Test 2 (Tight Tolerance): Setting the tolerance to 10^{-8} and comparing it to a tolerance of 10^{-4}, we see that the looser tolerance plateaus due to having a high error of margin. Since the tighter tolerance has an extremely small margin, the controller may partially overshoot and try to continually adjust itself over time, which leads to the oscillations. A tighter tolerance also requires around 100x more steps for a 10^{-4}x tighter tolerance.

Matthew Zeng
mz22535
April 30, 2026

Section 6: Conclusions

When Damping is Necessary: For linear ODEs, the full step (damping factor = 1) is essentially always sufficient. Damping becomes necessary for nonlinear ODEs or when the initial guess is extremely far from the actual root. However, implementing damping provides no risks or losses, so it is almost always better for a failsafe, robustness, or a nonlinear ODE.

When Adaptive Steps are Worth It: Using adaptive step sizes ends up paying off when the solution has multiple regions of contrasting behavior, such as our chemical system with a rapid transient reaction phase and a slowly varying equilibrium. In practice, smooth phases work well with large steps while more violent regions require smaller steps.

Lessons Learned: 1.) The safety factor and rate-of-change bounds are crucial since *h* may oscillate very drastically and create unwarranted step rejections. **2.)** The adaptive half step is much more accurate, and it is essentially always beneficial to use it since it doesn't cost anything and boosts overall accuracy of the solution.